



Data Warehouse and Business Intelligence

Tugas Kelompok: Multidimensional Modeling untuk Sistem DWBI Healthcare

Engelbertus Rande, Gilbert Fandiliam Mooy, Sinta Siti Nuriah, Vicky Mahfudy

**Dosen Pengampu
Khabib Mustofa**



Daftar Isi

I Pendahuluan 1
1 Pendahuluan 2
1.1 Deskripsi Domain 2
1.2 Latar Belakang 3
1.3 Tujuan 4
II Analisis Kebutuhan Bisnis 5
2 Analisis Kebutuhan Bisnis 6
2.1 Business Questions 6
2.2 Stakeholder 8
2.3 Proses Bisnis Utama 8
III Perancangan Dimensional Modeling 10
3 Perancangan Dimensional Modeling 11
3.1 Penentuan Grain 11
3.2 Rancangan Fact Table 12
3.3 Rancangan Dimension Table 12
3.4 Star Schema Diagram 13
3.5 Perancangan Arsitektur Staging dan Alur ETL 14
IV Synthetic Data Generation 17
4 Synthetic Data Generation 18
4.1 Strategi Generator Data Sintetik 18



4.2	Kode Generator Utama	18
4.3	Contoh Data Mentah (Output source_kunjungan.csv)	21
4.4	Skema Sumber (Data Dictionary)	21
V	Implementasi Orkestrasi Apache Airflow	22
5	Implementasi Orkestrasi Apache Airflow	23
5.1	Arsitektur DAG (Directed Acyclic Graph)	23
5.2	Topologi dan Penjelasan Task Dependencies	23
5.3	Panduan Dokumentasi Antarmuka Airflow UI	25

Part I

Pendahuluan



BAB 1

Pendahuluan

Anggota kelompok

1. Engelbertus Rande (NIM 25/573149/PPA/07211)
2. Gilbert Fandiliam Mooy (NIM 25/574767/PPA/07259)
3. Sinta Siti Nuriah (NIM 25/575177/PPA/07274)
4. Vicky Mahfudy (NIM 25/573019/PPA/07207)

1.1 DESKRIPSI DOMAIN

Domain yang menjadi fokus penelitian ini adalah bidang kesehatan (*healthcare*), khususnya pengelolaan data operasional rumah sakit yang mencakup registrasi pasien, penugasan tenaga medis, serta transaksi kunjungan ke fasilitas kesehatan. Domain ini bersifat multidimensional karena melibatkan berbagai aspek yang saling berkaitan, yaitu profil demografis pasien, klasifikasi keahlian dokter, serta rincian pelayanan medis baik di unit gawat darurat (IGD), rawat jalan, maupun rawat inap.

Populasi dalam dataset ini adalah populasi umum pasien yang melakukan kunjungan ke rumah sakit. Cakupan operasional dataset berfokus pada dinamika pelayanan dan administrasi penagihan (*billing*), di mana setiap kunjungan akan menghasilkan beban biaya yang berbeda bergantung pada tipe fasilitas, durasi perawatan, serta tindakan medis yang diberikan. Tiga aspek utama yang menjadi pilar analisis dalam domain ini adalah:

- **Kunjungan Pasien** — mencakup frekuensi penggunaan tipe fasilitas (rawat inap, rawat jalan, IGD), lama perawatan (*Length of Stay*), status pemulihan saat keluar, dan metode pembayaran.
- **Tenaga Medis** — mencakup data spesialisasi dokter, kepemilikan izin praktek (STR), dan struktur biaya konsultasi medis.
- **Rincian Biaya** — mencakup rincian penagihan yang terdiri dari biaya konsultasi, obat, tindakan, dan sewa kamar rawat inap yang kemudian diakumulasi menjadi total biaya transaksi.

Gambaran Umum Dataset

Berikut adalah karakteristik lengkap dataset yang digunakan dalam penelitian ini:



Tabel 1.1: Karakteristik Dataset Operasional Rumah Sakit

Aspek	Keterangan
Nama Dataset	Healthcare Operations Synthetic Dataset
Sumber Data 1	source_pasien.csv — Data profil demografis pasien (2.500 baris, 7 kolom)
Sumber Data 2	source_dokter.csv — Data master dokter (80 baris, 5 kolom)
Sumber Data 3	source_kunjungan.csv — Data transaksi layanan medis (12.000 baris, 13 kolom)
Total Record Kunjungan	12.000 baris
Rentang Waktu Kunjungan	1 Januari 2025 – Awal 2026
Tipe Fasilitas	IGD, Rawat Jalan, Rawat Inap
Metode Pembayaran	BPJS, Asuransi Swasta, Mandiri
Format Output	CSV (<i>Comma Separated Values</i>)

Dataset final didistribusikan ke dalam tiga entitas CSV independen yang saling berelasi. Pengelompokan atribut dan informasi kolom ditunjukkan pada tabel berikut.

Tabel 1.2: Pengelompokan Kolom pada 3 Tabel Sumber

Kelompok Kolom	Jml.	Contoh Kolom
Identitas & Kunci	3	id_pasien, id_dokter, id_kunjungan
Demografi Pasien	6	nama_pasien, jenis_kelamin, tanggal_lahir, alamat, kota, golongan_darah
Informasi Medis Dokter	4	nama_dokter, spesialisasi, biaya_konsultasi, nomor_izin_praktek
Utilisasi Layanan	4	tanggal_kunjungan, tipe_fasilitas, lama_rawat_hari, status_keluar
Finansial Medis	5	metode_pembayaran, biaya_obat, biaya_tindakan, biaya_kamar, total_biaya

1.2 LATAR BELAKANG

Manajemen operasional rumah sakit merupakan tantangan analitik terbesar di sistem pelayanan kesehatan. Transaksi operasional seringkali menghasilkan volume data klinis dan finansial yang sangat besar dari berbagai fasilitas (IGD, Poliklinik, dan Bangsal Inap). Fragmentasi data operasional di berbagai sistem pendaftaran, penjadwalan dokter, dan unit kasir menjadi hambatan utama dalam menghasilkan analitik keuangan dan utilitas fasilitas yang komprehensif.

Sistem Data Warehouse dan Business Intelligence (DWBI) hadir sebagai solusi untuk mengintegrasikan, menyimpan, dan menganalisis data transaksional dari berbagai sumber tersebut secara terstruktur dalam suatu model dimensional, sehingga menghasilkan wawasan operasional yang dapat ditindaklanjuti oleh pemangku kepentingan.

Penggunaan data sintesis dalam penelitian dan pengembangan DWBI ini dipilih untuk menghindari kendala regulasi perlindungan data privasi yang membatasi akses terhadap rekam medis nyata, namun tetap mempertahankan logika bisnis klinis operasional melalui teknik *synthetic data generation* berbasis aturan kondisional menggunakan bahasa pemrograman Python. Berikut merupakan identifikasi permasalahan:

1. Data pendaftaran pasien, jadwal dokter, dan *billing* layanan tersebar di berbagai sumber yang belum



terintegrasi, sehingga menyulitkan analisis performa rumah sakit secara holistik.

2. Belum tersedianya infrastruktur Data Warehouse yang mampu mengonsolidasikan ketiga aspek tersebut ke dalam satu skema *Star Schema* yang efisien untuk pelaporan bisnis.
3. Kurangnya kapabilitas untuk melacak rincian komponen biaya medis (obat, tindakan, kamar, jasa dokter) berdasarkan fasilitas dan spesialisasi secara *real-time*.
4. Keterbatasan akses terhadap data riil operasional akibat regulasi privasi memicu kebutuhan pembuatan *pipeline* generator data sintetik berbasis aturan medis (*rule-based*).

1.3 TUJUAN

Tujuan Umum

Membangun sistem Data Warehouse dan Business Intelligence (DWBI) operasional rumah sakit yang mengintegrasikan profil pasien, data master dokter, dan log transaksi kunjungan dari sumber terpisah ke dalam suatu pemodelan dimensional untuk mendukung pengambilan keputusan klinis dan finansial.

Tujuan Khusus

1. Mengintegrasikan data dari tiga sumber heterogen (`source_pasien.csv`, `source_dokter.csv`, dan `source_kunjungan.csv`) menjadi satu model data terpadu (12.000 transaksi).
2. Menganalisis pola utilisasi fasilitas layanan kesehatan (rawat inap, rawat jalan, IGD) dan mengevaluasi efisiensi operasional berbasis indikator durasi *Length of Stay* (LOS).
3. Menganalisis distribusi pendapatan operasional rumah sakit yang dirincikan berdasarkan komponen biaya, spesialisasi tenaga medis, dan metode pembayaran klaim.
4. Membangun skema dimensional (*Star Schema*) yang terfokus pada kemudahan ekstraksi pelaporan OLAP tingkat lanjut.
5. Menyediakan dataset sintetis berkualitas tinggi sebagai infrastruktur data untuk pengembangan, pengujian, dan demonstrasi sistem DWBI tanpa pelanggaran regulasi privasi pasien.

Part II

Analisis Kebutuhan Bisnis



BAB 2

Analisis Kebutuhan Bisnis

Anggota kelompok

1. Engelbertus Rande (NIM 25/573149/PPA/07211)
2. Gilbert Fandiliam Mooy (NIM 25/574767/PPA/07259)
3. Sinta Siti Nuriah (NIM 25/575177/PPA/07274)
4. Vicky Mahfudy (NIM 25/573019/PPA/07207)

2.1 BUSINESS QUESTIONS

Business questions merupakan pertanyaan-pertanyaan bisnis kritis yang diformulasikan berdasarkan kebutuhan analitik *stakeholder* manajemen rumah sakit serta karakteristik aktual dari struktur dataset operasional. Dua belas *business questions* berikut ditetapkan sebagai panduan utama dalam perancangan sistem DWBI, yang mencakup enam domain analitik utama: utilisasi fasilitas, kinerja tenaga medis, analisis finansial/penagihan, efisiensi klinis, metode pembayaran, dan demografi pasien.



Tabel 2.1: Business Questions Sistem DWBI Operasional Rumah Sakit

No	Business Question	Kategori	KPI / Metrik
1	Berapa total pendapatan rumah sakit dan bagaimana distribusinya berdasarkan tipe fasilitas (IGD, Rawat Jalan, Rawat Inap)?	Finansial	total_biaya, tipe_fasilitas
2	Spesialisasi dokter manakah yang menghasilkan volume transaksi kunjungan tertinggi dan total biaya penagihan terbesar?	Kinerja Dokter	Count(id_kunjungan), Sum(total_biaya) per spesialisasi
3	Bagaimana distribusi penggunaan metode pembayaran (BPJS, Asuransi Swasta, Mandiri) terhadap total biaya pelayanan?	Finansial & Payer	Persentase akumulasi total_biaya per metode_pembayaran
4	Berapa rata-rata lama rawat inap (<i>Length of Stay</i>) pasien dan bagaimana pengaruhnya terhadap biaya kamar serta total biaya transaksi?	Efisiensi Klinis	Rata-rata lama_rawat_hari, biaya_kamar, total_biaya
5	Bagaimana tren bulanan jumlah kunjungan pasien di rumah sakit sepanjang periode tahun 2025 hingga awal 2026?	Tren Kunjungan	Count(id_kunjungan) berdasarkan tren tanggal_kunjungan
6	Bagaimana proporsi status keluar pasien (Sembuh, Dirujuk, Pulang Paksa, Meninggal) khusus pada tipe fasilitas Rawat Inap?	Mutu Pelayanan	Rasio status_keluar where tipe_fasilitas = 'Rawat Inap'
7	Bagaimana rincian kontribusi masing-masing komponen biaya (konsultasi, obat, tindakan, kamar) terhadap struktur total penagihan?	Analisis Biaya	Breakdown rata-rata biaya_konsultasi, biaya_obat, biaya_tindakan, biaya_kamar
8	Kota asal pasien manakah yang memiliki beban pemanfaatan layanan dan frekuensi kunjungan tertinggi di rumah sakit?	Geografi	Count(id_kunjungan), Sum(total_biaya) per kota
9	Bagaimana profil distribusi kunjungan pasien jika dikelompokkan berdasarkan variabel jenis kelamin dan golongan darah?	Demografi	<i>Cross-tabulation</i> Count(id_kunjungan) per jenis_kelamin dan golongan_darah
10	Berapa rata-rata biaya obat yang dikeluarkan oleh pasien pada fasilitas IGD dibandingkan dengan fasilitas Rawat Jalan?	Farmasi & Biaya	Rata-rata biaya_obat per tipe_fasilitas (IGD vs Rawat Jalan)
11	Siapakah 5 dokter teratas (<i>Top 5 Doctors</i>) yang memiliki kontribusi pelayanan transaksi kunjungan paling aktif di rumah sakit?	Kinerja Dokter	Ranking 5 teratas nama_dokter berdasarkan Count(id_kunjungan)
12	Berapa rata-rata pengeluaran total biaya per satu kali transaksi kunjungan pasien (<i>average ticket size</i>) di rumah sakit?	Finansial	Rata-rata total_biaya dari seluruh baris transaksi

Business questions di atas dirancang agar dapat dijawab secara multidimensional melalui pembuatan tabel fakta transaksi (Fact_Kunjungan) yang akan terhubung dengan tabel-tabel dimensi pendukung seperti Dimensi Pasien, Dimensi Dokter, Dimensi Waktu, dan Dimensi Fasilitas pada arsitektur pemodelan data.



2.2 STAKEHOLDER

Identifikasi *stakeholder* diperlukan untuk memastikan sistem DWBI yang dibangun mampu menyajikan informasi yang relevan, akurat, dan sesuai dengan wewenang masing-masing pengguna di rumah sakit. Berdasarkan arsitektur data operasional yang tersedia, terdapat tujuh kelompok *stakeholder* utama:

Tabel 2.2: Daftar Stakeholder dan Kebutuhan Informasi

Stakeholder	Peran	Kebutuhan Informasi
Direktur / Manajemen RS	Decision Maker (Eksekutif)	Laporan KPI makro operasional, tren total pendapatan bulanan, efisiensi utilitas tempat tidur, dan evaluasi performa bisnis makro.
Kepala Bidang Pelayanan Medis	Clinical Manager	Monitoring beban kerja dokter berdasarkan spesialisasi, evaluasi hasil klinis pasien lewat rasio status keluar (Sembuh/Meninggal).
Manajer Keuangan / Akuntan	Financial Analyst	Analisis profitabilitas komponen penagihan (obat, tindakan, kamar), pemantauan perputaran arus kas penagihan rumah sakit.
Manajer Operasional & Fasilitas	Operational Analyst	Pengawasan utilitas tipe fasilitas (IGD, Rawat Jalan, Rawat Inap) dan analisis efisiensi operasional harian berdasarkan <i>Length of Stay</i> (LOS).
Unit Penjaminan & Klaim (Casemix)	Payer Analyst	Analisis segmentasi metode pembayaran (klaim BPJS, klaim Asuransi Swasta, dan Mandiri) untuk optimalisasi proses verifikasi penagihan.
Kepala Instalasi Farmasi RS	Pharmacy Supervisor	Evaluasi biaya obat yang terserap dalam transaksi pelayanan pasien di berbagai unit fasilitas (IGD dan Rawat Jalan).
Tim Data Engineer / IT RS	Sistem Administrator	Pemantauan kualitas data transaksi, optimasi performa <i>query</i> database OLAP, serta pemeliharaan <i>pipeline</i> ETL dari file CSV sumber.

Sistem DWBI ini menjembatani kebutuhan informasi dari tingkat eksekutif yang memerlukan data agregasi tinggi untuk keputusan strategis (seperti total pendapatan atau tren kunjungan), hingga tingkat operasional yang membutuhkan analisis granularitas rendah (seperti rincian biaya per transaksi).

2.3 PROSES BISNIS UTAMA

Enam proses bisnis utama berikut diidentifikasi sebagai rantai nilai operasional (*value chain*) yang menjadi sumber data transaksional dan konteks analitik utama di dalam lingkungan rumah sakit:



Tabel 2.3: Proses Bisnis Utama Domain Operasional Rumah Sakit

No	Proses Bisnis	Deskripsi	Kolom Terkait
1	Pendaftaran & Profil Pasien	Proses pencatatan data demografis, identitas unik, dan karakteristik fisik pasien saat pertama kali terdaftar di rumah sakit.	id_pasien, nama_pasien, jenis_kelamin, tanggal_lahir, alamat, kota, golongan_darah
2	Manajemen Data Master Dokter	Pengelolaan data ketenagakerjaan tenaga medis, legalitas izin praktek, penentuan spesialisasi, dan standarisasi tarif jasa dokter.	id_dokter, nama_dokter, spesialisasi, biaya_konsultasi, nomor_izin_praktek
3	Pelayanan Kunjungan Fasilitas	Proses penerimaan pasien di gerbang pelayanan (IGD, Rawat Jalan, Rawat Inap) pada waktu tertentu untuk dialokasikan ke dokter yang bertugas.	id_kunjungan, tanggal_kunjungan, tipe_fasilitas
4	Perawatan Rawat Inap (Opname)	Proses pemantauan durasi hari menginap pasien di bangsal perawatan serta pencatatan kondisi atau status keluar akhir pasien dari perawatan.	lama_rawat_hari, status_keluar
5	Perhitungan <i>Billing</i> & Biaya Medis	Proses kalkulasi penagihan otomatis yang mengakumulasikan komponen biaya penunjang medis (jasa dokter, konsumsi obat, dan biaya tindakan).	biaya_konsultasi, biaya_obat, biaya_tindakan, biaya_kamar
6	Penyelesaian Pembayaran (Klaim)	Proses pelunasan seluruh tagihan transaksi kunjungan pasien berdasarkan metode penjaminan yang disepati oleh unit kasir.	metode_pembayaran, total_biaya

Keenam proses bisnis di atas merepresentasikan siklus operasional harian rumah sakit yang terintegrasi. Setiap transaksi yang divalidasi oleh sistem akan terekam ke dalam file `source_kunjungan.csv` yang memuat relasi kunci asing ke `source_pasien.csv` dan `source_dokter.csv`. Pemahaman mendalam terhadap rantai proses bisnis ini menjadi fondasi utama dalam menentukan *grain*, *measures*, dan dimensi pendukung dalam perancangan model dimensional (*Star Schema*) pada bab berikutnya.

Part III

Perancangan Dimensional Modeling



BAB 3

Perancangan Dimensional Modeling

Anggota kelompok

1. Engelbertus Rande (NIM 25/573149/PPA/07211)
2. Gilbert Fandilam Mooy (NIM 25/574767/PPA/07259)
3. Sinta Siti Nuriah (NIM 25/575177/PPA/07274)
4. Vicky Mahfudy (NIM 25/573019/PPA/07207)

3.1 PENENTUAN GRAIN

Grain adalah pernyataan tegas mengenai apa yang direpresentasikan oleh satu baris data (*record*) di dalam tabel fakta (*fact table*). Penentuan *grain* yang tepat merupakan langkah paling kritis dalam perancangan model dimensional karena secara langsung akan menentukan *measures* apa saja yang dapat dihitung, dimensi apa saja yang diperlukan, serta tingkat kedalaman analisis yang dapat dieksplorasi.

Tabel 3.1: Definisi Grain Fact Table Operasional

Komponen	Definisi / Nilai
Subjek Ukur	Satu transaksi kunjungan pelayanan medis yang dialami pasien dari satu dokter pada satu tanggal tertentu
Granularitas Waktu	Tingkat harian / stempel waktu berdasarkan tanggal_kunjungan
Granularitas Pasien	Per pasien unik, diidentifikasi oleh kunci id_pasien
Granularitas Dokter	Per dokter unik, diidentifikasi oleh kunci id_dokter
Pernyataan Grain	Satu baris mewakili satu transaksi kunjungan medis seorang pasien ke suatu unit fasilitas yang ditangani oleh dokter tertentu pada tanggal tertentu, lengkap dengan rincian biaya penagihannya.
Implikasi	Satu pasien dapat memiliki banyak baris fakta jika melakukan kunjungan ulang ke rumah sakit pada tanggal atau fasilitas yang berbeda.

Grain pada tingkat transaksi per-kunjungan dipilih karena memberikan fleksibilitas analisis yang maksimal dari level makro (seperti laporan pendapatan total per bulan) hingga ke level mikro (seperti beban biaya obat per individu pasien di fasilitas IGD).



3.2 RANCANGAN FACT TABLE

Jenis Fact Table

Tabel fakta utama dalam sistem DWBI ini diberi nama **Fact_Kunjungan** dan tergolong sebagai **Transaction Fact Table**, di mana setiap baris mewakili satu kejadian (*event*) transaksi yang berdiri sendiri dan bersifat final ketika pasien telah melunasi penagihan dan keluar dari fasilitas rumah sakit.

Terdapat dua jenis *measures* di dalam tabel ini: (1) **Additive Measures** — nilai numerik yang dapat dijumlahkan secara logis ke seluruh dimensi, seperti komponen biaya medis dan nilai tagihan; serta (2) **Semi-Additive Measures** — nilai ukur yang hanya bisa diolah pada dimensi tertentu namun tidak masuk akal jika dijumlahkan pada dimensi waktu, seperti hari lama rawat inap (*length of stay*).

Daftar Measures dan Foreign Keys

Tabel 3.2: Rancangan Fact_Kunjungan (5 FK + 6 Measures)

ID	Nama Kolom	Tipe Measure	Tipe Data	Keterangan
FK1	pasien_key	Foreign Key	VARCHAR	Relasi ke entitas Dim_Pasien
FK2	dokter_key	Foreign Key	VARCHAR	Relasi ke entitas Dim_Dokter
FK3	date_key	Foreign Key	INT	Relasi penanggalan ke Dim_Waktu
FK4	fasilitas_key	Foreign Key	INT	Relasi ke Dim_Fasilitas
FK5	bayar_key	Foreign Key	INT	Relasi ke Dim_Pembayaran
M1	lama_rawat_hari	Semi-Additive	INT	Durasi opname pasien (0 jika selain Rawat Inap)
M2	biaya_konsultasi	Additive	DECIMAL	Beban tarif jasa pemeriksaan oleh dokter
M3	biaya_obat	Additive	DECIMAL	Beban persepan obat farmasi dari kunjungan
M4	biaya_tindakan	Additive	DECIMAL	Beban prosedur diagnostik atau tindakan operatif
M5	biaya_kamar	Additive	DECIMAL	Beban biaya sewa kasur unit inap per kelas
M6	total_biaya	Additive	DECIMAL	Akumulasi tagihan akhir pasien (M2+M3+M4+M5)

3.3 RANCANGAN DIMENSION TABLE

Jenis Slowly Changing Dimension (SCD)

Slowly Changing Dimension (SCD) adalah teknik pengarsipan yang digunakan untuk menangani perubahan atribut deskriptif dimensi seiring berjalannya waktu agar rekam jejak finansial di masa lalu tidak mengalami kerusakan data.



Tabel 3.3: Konfigurasi Tipe SCD pada Sistem DWBI

Tipe SCD	Nama Lain	Karakteristik	Contoh Kasus
Tipe 0	Retain / Fixed	Atribut tetap abadi dan tidak akan pernah diperbarui setelah pertama direkam.	Data kalender hari dan bulan pada Dim_Waktu
Tipe 1	Overwrite	Nilai lama langsung dihapus dan diganti nilai baru tanpa riwayat histori.	Perbaikan salah ketik ejaan nama / <i>update</i> izin STR pada Dim_Dokter
Tipe 2	Add New Row	Setiap pembaruan menghasilkan penambahan baris data baru beserta rentang penanda tanggal aktifnya.	Perubahan lokasi domisili / kota / status demografi pada Dim_Pasien

Rancangan Lima Dimension Table

Tabel 3.4: Rancangan Detail Tabel Dimensi

Dimensi	Tipe SCD	Primary Key	Atribut / Kolom Utama
Dim_Pasien	SCD Tipe 2	pasien_key	id_pasien, nama_pasien, jenis_kelamin, tanggal_lahir, alamat, kota, golongan_darah
Dim_Dokter	SCD Tipe 1	dokter_key	id_dokter, nama_dokter, spesialisasi, nomor_izin_praktek
Dim_Waktu	SCD Tipe 0	date_key	tanggal, nama_hari, bulan, kuartal, tahun, is_weekend_flag
Dim_Fasilitas	SCD Tipe 1	fasil_key	tipe_fasilitas (IGD, Rawat Inap, Rawat Jalan), status_keluar
Dim_Pembayaran	SCD Tipe 1	bayar_key	metode_pembayaran (BPJS, Swasta, Mandiri)

Dim_Pasien berperan sebagai dimensi subjek pelayanan yang menyimpan riwayat perpindahan domisili demografis menggunakan skema penambahan baris data baru (SCD Tipe 2). Sementara Dim_Waktu diturunkan sepenuhnya dari atribut operasional tanggal_kunjungan ke dalam struktur hierarki agregasi yang luas (hari ke minggu, bulan, dan tahun) secara permanen.

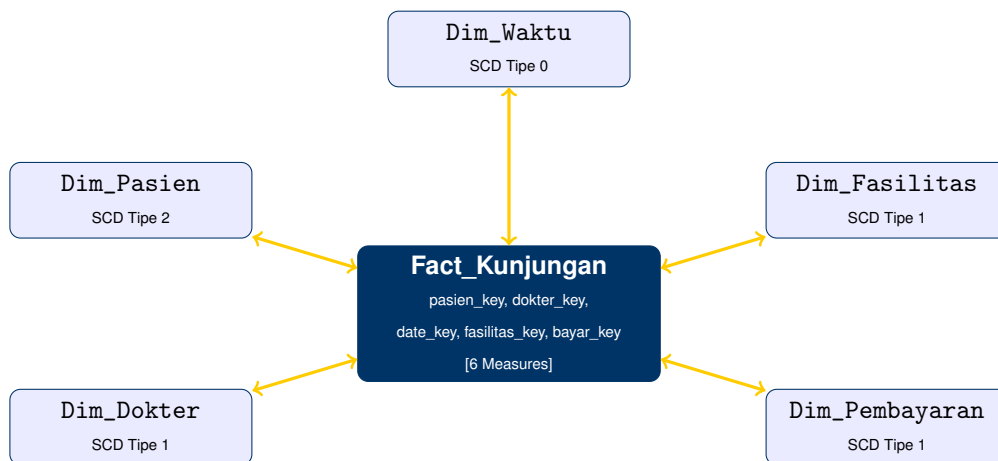
3.4 STAR SCHEMA DIAGRAM

Pendekatan topologi Data Warehouse diwujudkan melalui skema bintang (*Star Schema*). Arsitektur ini dipilih dibandingkan dengan skema hibrida seperti *Snowflake Schema* karena menawarkan kecepatan komputasi dan kesederhanaan eksekusi *query* OLAP melalui denormalisasi penuh ke arah tabel sentral.



Tabel 3.5: Pemetaan Komponen Arsitektur Star Schema

Nama Tabel	Jenis Model	Keterangan Arsitektur
Fact_Kunjungan	Fact Table	Inti perhitungan finansial dan kunjungan (5 kunci tamu + 6 kalkulasi numerik)
Dim_Pasien	Dimensi	Menjelaskan karakteristik pasien, terikat melalui pasien_key
Dim_Dokter	Dimensi	Menjelaskan identitas & spesialisasi medis, terikat melalui dokter_key
Dim_Waktu	Dimensi	Menentukan batasan agregasi <i>time-series</i> , terikat melalui date_key
Dim_Fasilitas	Dimensi	Memetakan pengelompokan jenis layanan unit, terikat via fasilitas_key
Dim_Pembayaran	Dimensi	Memetakan segmentasi arus kas/penjamin, terikat via bayar_key



Catatan Star Schema

Fact_Kunjungan berada di jantung arsitektur. Secara relasional, setiap rincian transaksi finansial tagihan medis diapit secara langsung (melalui konjungsi kunci indeks numerik tunggal) oleh penjelasan informatif dari lima dimensi pendukung: identitas Pasien yang dilayani, Dokter yang bertugas, Waktu kalender perawatan, letak Fasilitas penanganan, hingga model Pembayaran yang disepakati.

3.5 PERANCANGAN ARSITEKTUR STAGING DAN ALUR ETL

Untuk memastikan data transaksional dapat mengalir dari sistem sumber (file generator) ke dalam *Star Schema* dengan aman dan terstruktur, dirancang sebuah arsitektur *Extract, Transform, Load* (ETL) yang memanfaatkan *Staging Area*.

Konsep dan Fungsi Staging Area

Staging Area merupakan area penyimpanan sementara (*buffer zone*) yang menjembatani file CSV sumber dengan repositori akhir Data Warehouse. Penggunaan *staging area* dalam perancangan ini bertujuan untuk:

- **Ekstraksi Cepat:** Memindahkan data dari file `source_*.csv` ke dalam database relasional secepat



mungkin untuk meminimalisasi beban baca pada file sumber.

- **Transformasi & Standarisasi:** Memvalidasi format tipe data (seperti format tanggal dan angka desimal) sebelum disuntikkan ke skema utama.
- **Pembentukan Surrogate Key:** Tempat pemetaan kunci alami (*Natural Key* seperti *id_pasien*) menjadi kunci pengganti numerik (*Surrogate Key*) untuk mempercepat performa *query* OLAP.

Pemetaan Tabel Staging

Skema *staging* dirancang dengan prinsip relasi satu-banding-satu (1:1) terhadap file CSV yang dihasilkan oleh Python. Data akan dimuat dengan metode *Truncate and Load* ke dalam tiga tabel *staging* berikut:

Tabel 3.6: Pemetaan File Sumber ke Tabel Staging

Nama File Sumber	Tabel Staging	Karakteristik Pemindahan Data
source_pasien.csv	stg_pasien	Memuat profil pasien mentah; tipe data string dan date dipertahankan.
source_dokter.csv	stg_dokter	Memuat data master dokter beserta spesialisasi dan biaya dasar konsultasi.
source_kunjungan.csv	stg_kunjungan	Memuat seluruh 12.000 transaksi penagihan; bertindak sebagai tabel beban terberat.

Alur Transformasi ke Star Schema

Setelah data mentah mendarat di tabel *staging*, proses *Transform* dan *Load* akan mendistribusikan data tersebut ke dalam 5 Tabel Dimensi dan 1 Tabel Fakta dengan urutan logis sebagai berikut:

Langkah 1: Pengisian Tabel Dimensi

Tabel dimensi wajib diisi lebih dahulu untuk menjaga integritas *Foreign Key*.

- **Dim_Waktu:** Sistem mengekstrak nilai *distinct* tanggal_kunjungan dari stg_kunjungan, lalu memecahnya menjadi komponen hari, bulan, tahun, dan *weekend flag*.
- **Dim_Fasilitas & Dim_Pembayaran:** Sistem mengambil nilai unik (IGD/Inap/Jalan) dan metode pembayaran (BPJS/Mandiri/Swasta) dari stg_kunjungan.
- **Dim_Pasien & Dim_Dokter:** Data dari stg_pasien dan stg_dokter dipindahkan ke dimensi utama. Pada tahap ini, sistem menghasilkan *Surrogate Key* (pasien_key, dokter_key) dan mengevaluasi status perubahan data untuk SCD.

Langkah 2: Pengisian Tabel Fakta

Pembentukan tabel Fact_Kunjungan dieksekusi melalui *query* JOIN antara stg_kunjungan dengan kelima tabel dimensi yang telah diperbarui.

- Sistem mengambil metrik nominal (*Measures*) secara langsung: biaya_konsultasi, biaya_obat, biaya_tindakan, biaya_kamar, total_biaya, dan lama_rawat_hari.



- Sistem melakukan proses *Lookup* pada setiap dimensi untuk mengganti *Natural Key* (misal: PSN-00001) menjadi *Surrogate Key* numerik.
- Seluruh kombinasi *Surrogate Key* dan *Measures* disuntikkan sebagai baris baru ke dalam *Fact_Kunjungan*.

Orkestrasi Pipeline ETL

Proses pergerakan data dari file CSV ke *Staging* hingga ke *Star Schema* akan diotomatisasi secara terjadwal menggunakan Apache Airflow. Hal ini menjamin siklus pembaruan laporan DWBI di rumah sakit berjalan harian secara konsisten tanpa intervensi manual.

Part IV

Synthetic Data Generation



BAB 4

Synthetic Data Generation

4.1 STRATEGI GENERATOR DATA SINTETIK

Proses perancangan ekosistem data rumah sakit dalam skala volume 12.000 transaksi mensyaratkan strategi simulasi logis yang merepresentasikan probabilitas keadaan lapangan yang sesungguhnya (*Constraint Enforcement*). Metodologi augmentasi Python berlandaskan aturan berikut:

- **Sinergi Kredensial Tarif Medis:** Beban tarif pada biaya_konsultasi bukanlah angka yang diacak sepenuhnya, melainkan dikunci berdasarkan nilai kamus tarif spesialisasi.
- **Isolasi Logika Fasilitas Non-Inap:** Jika alokasi unit merujuk ke “IGD” atau “Rawat Jalan”, algoritma menginstruksikan kalkulasi durasi inap absolut di angka 0 dan tidak membebankan biaya sewa ruangan.
- **Pembobotan Pemulihan Pasien Inap:** Pasien yang dialokasikan ke “Rawat Inap” memiliki hari rawat yang disimulasikan (1–14 hari) dan luaran klinis yang ditentukan oleh probabilitas empiris: 80% dinyatakan Sembuh, dan hanya sisanya Dirujuk/Meninggal.
- **Logika Integritas Keuangan (*Billing*):** `total_biaya` dieksekusi secara matematis ketat melalui penjumlahan komponen linier: jasa dokter + obat farmasi + tindakan alat + sewa ruangan, meniadakan kesalahan agregasi fiskal pada Data Warehouse.

4.2 KODE GENERATOR UTAMA

Berikut adalah *source code* Python yang dikompilasi secara deterministik untuk merakit data sumber transaksi rumah sakit operasional ke dalam direktori file CSV:

Listing 4.1: Generator Data Sintetik Operasional Rumah Sakit

```
1 import os
2 import random
3 from datetime import datetime, timedelta
4 import pandas as pd
5 from faker import Faker
6
7 # Inisialisasi Faker dengan lokalisasi Indonesia
8 fake = Faker('id_ID')
9 Faker.seed(42)
10 random.seed(42)
11
```



```

12 # Konfigurasi Jumlah Data
13 TOTAL_PASIEN = 2500
14 TOTAL_DOKTER = 80
15 TOTAL_TRANSAKSI_KUNJUNGAN = 12000
16
17 print("Memulai pembuatan dataset sintetik rumah sakit...")
18
19 # =====
20 # 1. GENERATE TABEL SUMBER: PASIEN
21 # =====
22 data_pasien = []
23 golongan_darah_opsi = ['A', 'B', 'AB', 'O']
24 jenis_kelamin_opsi = ['Laki-laki', 'Perempuan']
25
26 for pasien_id in range(1, TOTAL_PASIEN + 1):
27     jk = random.choice(jenis_kelamin_opsi)
28     nama = fake.name_male() if jk == 'Laki-laki' else fake.name_female()
29     data_pasien.append({
30         'id_pasien'       : f'PSN-{pasien_id:05d}',
31         'nama_pasien'     : nama,
32         'jenis_kelamin'   : jk,
33         'tanggal_lahir'   : fake.date_of_birth(minimum_age=0, maximum_age=85)
34                             .strftime('%Y-%m-%d'),
35         'alamat'         : fake.street_address(),
36         'kota'           : fake.city(),
37         'golongan_darah' : random.choice(golongan_darah_opsi)
38     })
39 df_pasien = pd.DataFrame(data_pasien)
40
41 # =====
42 # 2. GENERATE TABEL SUMBER: DOKTER
43 # =====
44 data_dokter = []
45 spesialisasi_opsi = {
46     'Umum': 30, 'Anak': 40, 'Penyakit Dalam': 50,
47     'Bedah': 60, 'Jantung': 80, 'Saraf': 70, 'Mata': 45
48 }
49
50 for dokter_id in range(1, TOTAL_DOKTER + 1):
51     spesialisasi = random.choice(list(spesialisasi_opsi.keys()))
52     biaya_konsultasi = spesialisasi_opsi[spesialisasi] * 1000
53     data_dokter.append({
54         'id_dokter'       : f'DKT-{dokter_id:03d}',
55         'nama_dokter'     : fake.name(),
56         'spesialisasi'     : spesialisasi,
57         'biaya_konsultasi' : biaya_konsultasi,
58         'nomor_izin_praktek' : f'STR-{random.randint(100000, 999999)}'
59     })
60 df_dokter = pd.DataFrame(data_dokter)
61
62 # =====
63 # 3. GENERATE TABEL SUMBER: KUNJUNGAN
64 # =====
65 data_kunjungan = []
66 tipe_fasilitas_opsi = ['IGD', 'Rawat Jalan', 'Rawat Inap']
67 status_keluar_opsi = ['Sembuh', 'Dirujuk', 'Pulang Paksa', 'Meninggal']
68 metode_pembayaran_opsi = ['BPJS', 'Asuransi Swasta', 'Mandiri']

```



```

69 start_date = datetime(2025, 1, 1)
70
71 for kunjungan_id in range(1, TOTAL_TRANSAKSI_KUNJUNGAN + 1):
72     pasien = random.choice(data_pasien)
73     dokter = random.choice(data_dokter)
74     tipe_fasilitas = random.choice(tipe_fasilitas_opsi)
75     metode_pembayaran = random.choice(metode_pembayaran_opsi)
76
77     # Logika Lama Rawat Inap (Length of Stay)
78     if tipe_fasilitas == 'Rawat Inap':
79         lama_rawat = random.randint(1, 14)
80         status_keluar = random.choices(
81             status_keluar_opsi, weights=[80, 12, 6, 2])[0]
82         biaya_kamar = lama_rawat * random.choice([250000, 500000,
83             1200000])
84     else:
85         lama_rawat = 0
86         status_keluar = 'Sembuh'
87         biaya_kamar = 0
88
89     # Simulasi Komponen Biaya Medis
90     biaya_konsul = dokter['biaya_konsultasi']
91     biaya_obat = random.randint(50000, 750000)
92     if tipe_fasilitas != 'IGD':
93         else random.randint(30000, 200000)
94     biaya_tindakan = random.choice([0, 150000, 300000, 1500000])
95     if tipe_fasilitas in ['IGD', 'Rawat Inap'] else 0)
96     total_biaya = biaya_konsul + biaya_obat + biaya_tindakan +
97         biaya_kamar
98     tgl_kunjungan = start_date + timedelta(days=random.randint(0, 450))
99
100     data_kunjungan.append({
101         'id_kunjungan' : f'TX-{kunjungan_id:06d}',
102         'id_pasien' : pasien['id_pasien'],
103         'id_dokter' : dokter['id_dokter'],
104         'tanggal_kunjungan' : tgl_kunjungan.strftime('%Y-%m-%d %H:%M:%S'),
105         'tipe_fasilitas' : tipe_fasilitas,
106         'lama_rawat_hari' : lama_rawat,
107         'status_keluar' : status_keluar,
108         'metode_pembayaran' : metode_pembayaran,
109         'biaya_konsultasi' : biaya_konsul,
110         'biaya_obat' : biaya_obat,
111         'biaya_tindakan' : biaya_tindakan,
112         'biaya_kamar' : biaya_kamar,
113         'total_biaya' : total_biaya
114     })
115
116 df_kunjungan = pd.DataFrame(data_kunjungan)
117
118 # =====
119 # 4. MENYIMPAN DATA KE CSV
120 # =====
121 OUTPUT_DIR = '/opt/airflow/data_generator/output'
122 os.makedirs(OUTPUT_DIR, exist_ok=True)
123
124 df_pasien.to_csv(os.path.join(OUTPUT_DIR, 'source_pasien.csv'), index=False)
125 df_dokter.to_csv(os.path.join(OUTPUT_DIR, 'source_dokter.csv'), index=False)
126 df_kunjungan.to_csv(os.path.join(OUTPUT_DIR, 'source_kunjungan.csv'), index=

```



```

False)
124
125 print("Pembuatan dataset selesai!")
126 print(f"    - Total Pasien    : {len(df_pasien)} baris")
127 print(f"    - Total Dokter    : {len(df_dokter)} baris")
128 print(f"    - Total Kunjungan: {len(df_kunjungan)} baris")

```

4.3 CONTOH DATA MENTAH (OUTPUT SOURCE_KUNJUNGAN.CSV)

Berikut adalah cuplikan simulasi hasil output transaksi yang memperlihatkan keutuhan integritas perhitungan *billing* matematis dalam *pipeline* DWBI ini.

Tabel 4.1: Contoh Data Mentah Transaksi Kunjungan

ID Kunjun- gan	Fasilitas	LOS	Biaya Kon- sul	Biaya Obat	Biaya Tin- dakan	Biaya Ka- mar	Total (Rp)
TX-000001	Rawat Inap	4	50.000	630.000	300.000	1.000.000	1.980.000
TX-000002	IGD	0	80.000	75.000	1.500.000	0	1.655.000
TX-000003	Rawat Jalan	0	30.000	125.000	0	0	155.000

4.4 SKEMA SUMBER (DATA DICTIONARY)

Skema akhir luaran generator diklasifikasikan menjadi tiga repositori utama yang saling terhubung menggunakan *Primary Key* (PK) dan *Foreign Key* (FK).

Sumber 1 — source_pasien.csv

Data master yang berisi informasi profil identitas pasien, meliputi: *id_pasien* (PK), *nama_pasien*, *jenis_kelamin*, *tanggal_lahir*, *alamat*, *kota*, dan *golongan_darah*.

Sumber 2 — source_dokter.csv

Data master yang berisi parameter profesi kesehatan dokter, meliputi: *id_dokter* (PK), *nama_dokter*, *spesialisasi*, *biaya_konsultasi*, dan *nomor_izin_praktek*.

Sumber 3 — source_kunjungan.csv

Data transaksi historis operasional yang menautkan kunci referensial dan parameter beban tagihan pasien, meliputi: *id_kunjungan* (PK), *id_pasien* (FK), *id_dokter* (FK), *tanggal_kunjungan*, *tipe_fasilitas*, *lama_rawat_hari*, *status_keluar*, *metode_pembayaran*, *biaya_konsultasi*, *biaya_obat*, *biaya_tindakan*, *biaya_kamar*, dan *total_biaya*.

Part V

Implementasi Orkestrasi Apache Airflow



BAB 5

Implementasi Orkestrasi Apache Airflow

5.1 ARSITEKTUR DAG (DIRECTED ACYCLIC GRAPH)

Apache Airflow bertindak sebagai mesin pengatur dan penjadwal utama (*enterprise orchestrator*) untuk mengotomatisasi seluruh siklus data dalam sistem DWBI rumah sakit ini. Di dalam Airflow, rangkaian alur kerja didefinisikan sebagai **Directed Acyclic Graph (DAG)** bernama `02_healthcare_data_pipeline`. Karakteristik "*directed*" menandakan alur eksekusi memiliki arah ketergantungan yang jelas dari awal hingga akhir, sedangkan "*acyclic*" menjamin bahwa tidak ada tugas yang berjalan berulang atau membentuk putaran tak berujung (*infinite loop*).

Konfigurasi parameter operasional dan manajemen risiko dari DAG ini dirancang menggunakan argumen bawaan (*default args*) sebagai berikut:

- **Owner** (`kelompok_healthcare`): Digunakan sebagai identitas pengelola aset *pipeline* untuk mempermudah kategorisasi, audit log, dan pemfilteran pada dasbor Web UI Airflow.
- **Depends on Past (False)**: Keberhasilan eksekusi *pipeline* hari ini tidak digantungkan pada status keberhasilan eksekusi hari sebelumnya, sehingga mencegah kemacetan massal (*data blocking*).
- **Retries (1) & Retry Delay (2 menit)**: Jika sebuah *task* gagal akibat masalah jaringan atau database PostgreSQL yang sedang sibuk, Airflow akan menjadwalkan ulang eksekusi otomatis satu kali lagi setelah jeda 2 menit.
- **Schedule Interval (None) & Catchup (False)**: *Pipeline* dikonfigurasi untuk berjalan berdasarkan pemicu manual (*Manual Trigger*). Penonaktifan *catchup* mencegah Airflow melakukan eksekusi susulan (*backfilling*) terhadap jadwal historis masa lalu.

5.2 TOPOLOGI DAN PENJELASAN TASK DEPENDENCIES

Pipeline dipecah menjadi 4 *task* diskrit yang dikelola oleh dua operator utama: `BashOperator` (untuk mengeksekusi skrip eksternal di lingkungan OS) dan `PythonOperator` (untuk mengeksekusi fungsi di dalam memori Python). Keempat *task* diikat menggunakan operator dependensi sekuensial linear (`»`), membentuk urutan kronologis sebagai berikut:

Task 1: `run_data_generator` (**BashOperator**)



Airflow memicu lingkungan *shell* Linux untuk menjalankan skrip pembuat data sintetik.

Listing 5.1: Definisi Task `run_data_generator`

```
1 task_generate_data = BashOperator(
2     task_id='run_data_generator',
3     bash_command='python /opt/airflow/data_generator/src/generator.py',
4 )
```

Airflow memastikan file `generator.py` selesai dieksekusi 100% dan menghasilkan 3 file CSV di folder *shared storage* sebelum mengizinkan aliran data berpindah ke tahap berikutnya.

Task 2: `ingest_csv_to_database` (PythonOperator)

Airflow memanggil fungsi Python internal untuk memindahkan data dari file `.csv` ke tabel relasional database.

Listing 5.2: Definisi Task `ingest_csv_to_database`

```
1 task_ingest_data = PythonOperator(
2     task_id='ingest_csv_to_database',
3     python_callable=ingest_csv_to_postgres,
4 )
```

Fungsi `ingest_csv_to_postgres` menggunakan *connection string* PostgreSQL `postgresql+psycopg2://airflow:airflow@localhost:5432/airflow` untuk membaca CSV via *Dataframe* Pandas dan menuliskannya ke tabel `src_pasien`, `src_dokter`, dan `src_kunjungan` dengan metode *overwrite* (`if_exists='replace'`).

Task 3: `dbt_run_transformation` (BashOperator)

Airflow memerintahkan `dbt` sebagai *third-party tool* untuk memproses transformasi.

Listing 5.3: Definisi Task `dbt_run_transformation`

```
1 task_dbt_run = BashOperator(
2     task_id='dbt_run_transformation',
3     bash_command=(
4         'cd /opt/airflow/dbt_project && '
5         'dbt run --target docker_env --profiles-dir .'
6     ),
7 )
```

Airflow mengirimkan sinyal perintah CLI `dbt run` untuk memproses pembentukan *Star Schema* (Tabel Fakta dan Dimensi) di dalam database.

Task 4: `dbt_data_quality_test` (BashOperator)

Airflow mengeksekusi proses pengujian kualitas data sebelum laporan siap dikonsumsi.

Listing 5.4: Definisi Task `dbt_data_quality_test`

```
1 task_dbt_test = BashOperator(
2     task_id='dbt_data_quality_test',
3     bash_command=(
4         'cd /opt/airflow/dbt_project && '
5         'dbt test --target docker_env --profiles-dir .'
6     ),
7 )
```



Jika pengujian integritas (seperti deteksi *null value* atau duplikasi kunci utama) menemukan kegagalan, Airflow akan menandai *task* ini berwarna merah (*Failed*) dan menghentikan seluruh *pipeline*.

5.3 PANDUAN DOKUMENTASI ANTARMUKA AIRFLOW UI

Untuk membuktikan bahwa arsitektur orkestrasi ini berjalan dengan sukses di lingkungan Docker, lampirkan beberapa tangkapan layar dari Apache Airflow Web UI (akses via `localhost:8080`) dengan penjelasan sebagai berikut:

- **Screenshot Graph View:** Buka dasbor DAG `02_healthcare_data_pipeline`, pilih tab *Graph*. Tangkap layar seluruh diagram kotak alur kerja. Seluruh bingkai kotak (*node*) harus berwarna hijau tua (*dark green*) sebagai indikator seluruh *script* sukses dieksekusi tanpa interupsi.
- **Screenshot Grid / Tree View:** Klik tab *Grid* di dasbor Airflow. Ambil gambar matriks kotak-kotak kecil historis eksekusi. Lajur kotak vertikal berwarna hijau penuh menandakan keberhasilan *run* secara utuh. Melalui tampilan ini, pengelola Data Warehouse dapat memantau durasi performa kecepatan eksekusi tiap *task* dan mendeteksi *bottleneck*.
- **Screenshot Task Log:** Klik salah satu kotak *task*, misalnya `run_data_generator` atau `ingest_csv_to_database`, lalu klik tombol *Log*. Ambil tangkapan layar baris teks keluaran terminal (*stdout*). Dokumen log menunjukkan status keluaran sistem yang bersih dengan kode balik `INFO - Main task process exited with return code 0`, membuktikan Airflow berhasil menyelesaikan pemanggilan fungsi eksternal dengan sukses.



5.3 DAFTAR PUSTAKA

- [1] Apache Software Foundation. Apache airflow documentation. URL <https://airflow.apache.org/docs/>.
- [2] dbt Labs, Inc. What is dbt? | dbt documentation. URL <https://docs.getdbt.com/docs/introduction>.
- [3] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. John Wiley & Sons, Inc., 2nd edition, 2002. ISBN 0-471-20024-7. URL http://www.r-5.org/files/books/computers/databases/warehouses/Ralph_Kimball_Margy_Ross-The_Data_Warehouse_Toolkit-EN.pdf.
- [4] PostgreSQL Global Development Group. *PostgreSQL 18.0 Documentation*. PostgreSQL Global Development Group, 2025. URL <https://www.postgresql.org/files/documentation/pdf/18/postgresql-18-A4.pdf>.